

TASK 2 SUBMISSION

Requirement Analysis & System Design

NBK Travel — Shuttle Booking Management System

Client	NBK Travel (nbktravel.co.za)
System	Shuttle Booking Management System
Module Code	XISD5319
Submission Date	April 2026
GitHub Repository	github.com/mzamon/Shuttle_Management_System

TEAM MEMBERS

Full Name	Student Number	Role
Shenice Wood	ST10447209	Project Manager / Team Leader
Murendeni Makhavhu	ST10377430	Database Administrator
Thandiwe Sibeko	ST10446961	System Analyst
Matome Maopye	ST10341694	UI/UX Designer & QA Tester
Mzamo Richmond Ndlovu	ST10455453	Software Developer

Document A

Requirement Analysis

NBK Travel Shuttle Booking Management System

Table of Contents — Requirement Analysis

1. Introduction / Problem Domain	3
1.1 Organisational Background	3
1.2 Problem Statement	3
1.3 Project Objectives	4
1.4 Stakeholders	4
1.5 Scope and Constraints	5
2. Solution Domain	5
2.1 Functional Requirements Table	5
2.2 UML Use Case Diagram	7
2.2.1 Actor Descriptions	7
3. Logical System Model	8
3.1 System Model Table	8
3.2 Method Naming Convention	10
4. Class Diagrams	10
4.1 Class Identification Table	10
4.2 Domain Class Diagram	13
Appendix A — UML Use Case Diagram	14
Appendix B — Domain Class Diagram	14
Appendix C — References	15

Requirement Analysis — NBK Travel Shuttle Booking Management System

1. Introduction / Problem Domain

This section presents a complete study of the problem domain identified within **NBK Travel** (nbktravel.co.za), a shuttle and transport service company operating across South Africa. The analysis expands upon the introduction provided in the Task 1 Project Plan and delivers a full specification of the operational challenges, system requirements, and project constraints that the Shuttle Booking Management System is designed to address (IIE, 2026).

1.1 Organisational Background

NBK Travel is a South African shuttle and transport services company providing airport transfers, corporate travel, and general commuter transport. The company operates a fleet of vehicles across multiple routes and employs a team of professional drivers to serve both individual and corporate clients. NBK Travel's brand identity is built on reliability, punctuality, and personalised service — values that its current manual booking processes directly undermine.

Prior to this development initiative, all booking operations were conducted through telephone calls, paper-based registries, and personal spreadsheets. This approach creates critical inefficiencies and competitive disadvantages in an increasingly digital marketplace. As client volumes grow, the manual system becomes progressively less sustainable, threatening both operational integrity and NBK Travel's reputation in the South African transport sector.

The Shuttle Booking Management System has been commissioned to digitise and centralise NBK Travel's operational workflows through a web-based platform built using HTML, CSS, and JavaScript on the frontend, with a PHP backend and a MySQL relational database managed via phpMyAdmin on a WAMP Server environment (IIE, 2026).

1.2 Problem Statement

The following operational deficiencies were documented during requirements elicitation and stakeholder engagement with NBK Travel management (IIE, 2026):

- **Double-bookings and scheduling conflicts:** Without a real-time centralised registry, NBK Travel cannot detect simultaneous reservations for the same driver or vehicle, resulting in service failures and reputational damage.
- **Fragmented customer records:** Customer histories are dispersed across paper files and personal spreadsheets, making repeat-client recognition, preference retrieval, and proactive communication practically impossible.
- **Absence of business intelligence:** Management cannot generate accurate reports on trip volumes, revenue trends, driver utilisation rates, or route profitability, severely constraining both operational and strategic decision-making.
- **No digital invoicing capability:** Customer-facing invoices are produced manually, reducing professionalism and complicating dispute resolution.
- **No structured notification system:** Booking confirmations and driver reminders are communicated informally, leading to missed trips and a poor client experience.
- **No driver visibility dashboard:** Drivers have no centralised view of their assigned trips, leading to schedule confusion and missed assignments.

1.3 Project Objectives

The Shuttle Booking Management System will deliver the following specific, measurable outcomes:

- **Booking Management Module:** A digital system to create, store, modify, and retrieve all trip bookings with real-time availability verification.
- **Scheduling System:** Automated assignment of drivers and vehicles to confirmed bookings, with built-in conflict detection that blocks overlapping assignments.
- **Customer Database:** Comprehensive records for new, potential, and repeat clients — including contact details, booking history, and travel preferences.
- **Reporting Module:** On-demand reports covering trips per day and week, revenue summaries, driver utilisation rates, and route performance metrics.
- **Invoice Generator:** Automated PDF invoice production using html2pdf.js, ensuring professional, consistent billing.
- **Notification Simulation:** System-triggered booking confirmations and driver reminders via simulated email/SMS channels.
- **Driver Dashboard:** A dedicated view for drivers to see assigned trips, departure times, and pickup/drop-off details.

1.4 Stakeholders

Table 1.1: Project Stakeholders

Stakeholder	Role in System	Primary Interest
NBK Travel Management	Commission, approve, and use reports	Operational oversight, revenue tracking
NBK Travel Administrator	Primary system user — bookings, scheduling, reports	Efficiency, accuracy, reduced manual workload
NBK Travel Drivers	View assigned trips, mark trips complete	Clear schedule visibility
NBK Travel Customers	Submit requests, receive invoices and confirmations	Ease of booking, reliable service
Development Team	Build, test, and deliver the system	Academic credit, technical skill development
Rosebank College (IIE)	Academic assessor	WIL module learning outcomes

1.5 Scope and Constraints

In Scope (Version 1.0): Web-based booking management (CRUD) · Driver and vehicle scheduling with conflict detection · Customer record management and search · Trip and revenue reporting via Chart.js · PDF invoice generation via html2pdf.js · Simulated notification logging · Driver dashboard view	Out of Scope (Version 1.0): Live GPS vehicle tracking · Real payment gateway integration · Multi-branch or multi-fleet management · Native mobile application
---	---

Technical stack: HTML5/CSS3/JavaScript (frontend) | PHP 8.x + MySQLi (backend) | MySQL via phpMyAdmin (database)
| WAMP Server (hosting) | GitHub for version control.

2. Solution Domain

A logical description of all functional requirements of the proposed NBK Travel Shuttle Booking Management System is provided below. Requirements are mapped using the standard three-column format: active actor, system function, and passive actor. The UML Use Case Diagram is presented in Appendix A (IIE, 2026).

2.1 Functional Requirements Table

Table 2.1: Functional Requirements — NBK Travel Shuttle Booking Management System (22 requirements)

Participant (Active Actor)	Sub-System	Function of the System	Participant (Passive Actor)
Administrator	Booking Management	Create a new booking capturing customer name, pickup/drop-off location, date, time, number of passengers, and fare amount	Customer
Administrator	Booking Management	View all bookings in a filterable, paginated table sorted by date and status	—
Administrator	Booking Management	Edit an existing booking to update trip details or reschedule the date and time	Customer
Administrator	Booking Management	Cancel a booking and log a mandatory cancellation reason to the bookings table	Customer
Customer	Booking Management	Submit a shuttle booking request via the public-facing web form	Administrator
Administrator	Scheduling System	Assign an available driver and an available vehicle to a confirmed booking	Driver
System (Automated)	Scheduling System	Detect and flag overlapping driver or vehicle assignments automatically upon every assignment attempt	Administrator
Administrator	Scheduling System	View the daily and weekly schedule grid for all active drivers and vehicles	Driver
Driver	Scheduling System	View all personally assigned trips via the driver dashboard	Administrator
Driver	Scheduling System	Mark an assigned trip as completed upon passenger delivery	Administrator
Administrator	Customer Records	Add a new customer profile capturing full name, phone number, email address, and travel preferences	Customer
Administrator	Customer Records	Edit an existing customer profile to update contact details or preferences	Customer
Administrator	Customer Records	Search for a customer by name or phone number and view their full booking history	Customer
System (Automated)	Customer Records	Automatically match a new booking to an existing customer record based on name and phone number	Customer
Administrator	Reporting Module	Generate a trip count report grouped by day, week, or month for a selected date range	—
Administrator	Reporting Module	Generate a revenue summary report displaying total fare income for a selected period	—
Administrator	Reporting Module	View a ranked top customers report ordered by total booking count and total spend	Customer
Administrator	Reporting Module	Generate a driver utilisation report showing trips and hours per driver in a selected period	Driver
Administrator	Invoice Generator	Generate and download a PDF invoice for a completed booking using html2pdf.js	Customer
System (Automated)	Invoice Generator	Auto-populate the invoice with booking details, fare amount, tax calculation, and NBK Travel company information	Customer
System (Automated)	Notification Simulation	Log a simulated SMS and email booking confirmation when a booking is created or updated	Customer
System (Automated)	Notification Simulation	Log a simulated SMS and email departure reminder to the assigned driver before the trip departure time	Driver

2.2 UML Use Case Diagram

The UML Use Case Diagram for the NBK Travel Shuttle Booking Management System is presented in **Appendix A (Figure A.1)**. The diagram depicts the system boundary, six subsystems, all identified use cases (UC01–UC14), and four actors.

2.2.1 Actor Descriptions

Table 2.2: Actor Descriptions

Actor	Type	Description
Administrator	Active	NBK Travel's primary system user. Responsible for booking creation and management, scheduling, customer records, report generation, and invoice production. Full system access.
Driver	Active	NBK Travel shuttle driver. Views personally assigned trips via the driver dashboard and marks trips as completed. Read-only access to own schedule only.
Customer	Passive	NBK Travel client. Submits booking requests via the public web form and receives invoices and notifications generated by the system. No direct system login in Version 1.0.
System (Automated)	Active	The PHP backend automated process. Initiates conflict detection, customer record matching, invoice auto-population, and notification simulation without direct human input.

Use Case Relationships:

- UC03 (Assign Driver & Vehicle) «includes» UC04 (Detect Conflict) — conflict detection executes automatically on every assignment attempt.
- UC07 (Manage Customer Profile) «extends» UC08 (View Booking History) — history viewing is an optional extension of profile management.
- UC09 (Trip Report) «extends» UC10 (Revenue Report) — revenue reporting extends the base trip reporting capability.
- UC13 (Generate Invoice PDF) «includes» UC14 (Log Notification) — every invoice generation triggers a system notification log entry.

Figure A.1 — UML Use Case Diagram (see Appendix A)

Actors: Administrator · Driver · Customer · System (Automated)
Use Cases: UC01–UC14 across 6 subsystems
Relationships: «include» (UC03→UC04, UC13→UC14) | «extend» (UC07→UC08, UC09→UC10)
Diagram produced in draw.io — stored at /docs/diagrams/use-case-diagram.drawio
GitHub: https://github.com/mzamon/Shuttle_Management_System/

3. Logical System Model

The Logical System Model defines the precise mapping between user interface actions (GUI inputs), the PHP system methods invoked in response, the underlying MySQL database tables affected, and the resulting outputs. This table is the operational heart of the system specification — every row traces a complete interaction path from user action through to the data layer (IIE, 2026).

3.1 System Model Table

Table 3.1: Logical System Model — NBK Travel Shuttle Booking Management System

GUI Input	System Process (PHP Method)	Entity Relationship (MySQL Table)	GUI Output
Enter booking details (customer, pickup, drop-off, date, time, passengers, fare)	createBooking(\$customerId, \$pickup, \$dropoff, \$datetime, \$pax, \$fare)	bookings → customers (FK: customerId)	Booking confirmation card displayed; notification triggered
Select booking from the bookings list	getBookingById(\$bookingId)	bookings table	Full booking detail view rendered
Update booking fields and submit	updateBooking(\$bookingId, \$fields)	bookings table	Updated record confirmation; notification triggered
Click cancel, enter cancellation reason	cancelBooking(\$bookingId, \$reason)	bookings table	Cancellation status logged; booking status set to 'cancelled'
Select driver and vehicle for a booking	assignDriverVehicle(\$bookingId, \$driverId, \$vehicleId)	schedules → drivers (FK) → vehicles (FK)	Schedule updated; conflict modal shown if overlap detected
No input (triggered on every assignment attempt)	detectConflict(\$driverId, \$vehicleId, \$datetime, \$duration)	schedules table	Conflict warning displayed in red modal; assignment blocked if conflict = true
Select date or week for schedule view	getSchedule(\$startDate, \$endDate)	schedules → bookings (JOIN)	Daily or weekly calendar grid rendered
Driver clicks Mark Complete button	completeTrip(\$bookingId, \$driverId)	bookings → schedules	Trip status updated to 'completed'; driver dashboard refreshed
Enter customer details and submit	createOrUpdateCustomer(\$name, \$phone, \$email, \$preferences)	customers table	Customer profile saved; confirmation displayed
Type name or phone in search box	searchCustomer(\$query)	customers → bookings (JOIN)	Customer profile card and full booking history list displayed
No input (triggered on booking creation)	matchCustomerRecord(\$name, \$phone)	customers → bookings	Booking automatically linked to existing customer record
Select report period (date range) and group by	generateTripReport(\$start, \$end, \$groupBy)	bookings table (bookingDate, status)	Trip count Chart.js bar/line chart and printable HTML table
Select revenue period (date range)	generateRevenueReport(\$start, \$end)	bookings.fareAmount grouped by date	Revenue summary Chart.js bar chart and printable table
No input (top N customers)	getTopCustomers(\$limit)	customers → bookings (COUNT JOIN)	Ranked top customers HTML table with booking count and total spend
Select driver and period	getDriverStats(\$driverId, \$start, \$end)	schedules → bookings (WHERE driverId)	Driver utilisation HTML table showing trips and total hours
Select a completed booking	generateInvoice(\$bookingId)	bookings → customers → invoices	PDF invoice downloaded via html2pdf.js
No input (triggered on booking and invoice events)	logNotification(\$type, \$recipientId, \$msg, \$channel)	notifications table	Notification log entry written; displayed in audit log table
Driver logs in to dashboard	getAssignedTrips(\$driverId)	schedules → bookings (WHERE driverId)	Driver's assigned trips list rendered on driver dashboard
Administrator or driver submits login form	authenticateUser(\$username, \$password)	users table	PHP session started; role-appropriate dashboard rendered

3.2 Method Naming Convention

All PHP system methods follow a consistent camelCase verb-entity naming convention: **[verb][Entity](\$parameters)**. The verb describes the action performed (create, get, update, cancel, detect, generate, log, assign) and the entity identifies the primary MySQL table affected. Methods map directly to SQL operations — SELECT, INSERT, UPDATE, DELETE — executed through MySQLi prepared statements. This convention was agreed during the System Design phase (T3) to ensure consistency across all team member contributions (IIE, 2026).

4. Class Diagrams

Class diagrams model the static structure of the NBK Travel Shuttle Booking Management System. Each entity class maps directly to a MySQL database table implemented via phpMyAdmin. Properties include attributes with their MySQL data types. Relationships between classes reflect the foreign key relationships defined in the database schema. The domain class diagram is presented in **Appendix B (Figure B.1)** (IIE, 2026).

4.1 Class Identification Table

Table 4.1: Class Identification — NBK Travel Shuttle Booking Management System (8 classes)

Entity (UML Class)	Properties / Attributes (MySQL Data Types)	Related To
«abstract» Person	fullName : VARCHAR(100) phoneNumber : VARCHAR(20) emailAddress : VARCHAR(100)	Customer (inherits) Driver (inherits)
Customer (extends Person)	customerId : INT (PK, AUTO_INCREMENT) fullName : VARCHAR(100) phoneNumber : VARCHAR(20) emailAddress : VARCHAR(100) preferences : VARCHAR(255) createdAt : DATETIME Methods: createCustomer() · updateCustomer() · searchCustomer() · getBookingHistory() · getTopCustomers()	Booking (One-to-Many) Invoice (One-to-Many) Notification (One-to-Many)
Driver (extends Person)	driverId : INT (PK, AUTO_INCREMENT) fullName : VARCHAR(100) licenceNumber : VARCHAR(30) phoneNumber : VARCHAR(20) status : ENUM('available','on-trip','off-duty') Methods: assignToBooking() · markTripComplete() · getAssignedTrips() · getSchedule() · getDriverStats()	Booking (One-to-Many) Schedule (One-to-Many) Notification (One-to-Many)
Booking	bookingId : INT (PK, AUTO_INCREMENT) customerId : INT (FK → customers) driverId : INT (FK → drivers) vehicleId : INT (FK → vehicles) pickupLocation : VARCHAR(100) dropoffLocation : VARCHAR(100) bookingDate : DATETIME passengers : INT status : ENUM('pending','confirmed','completed','cancelled') cancellationReason : VARCHAR(255) fareAmount : DECIMAL(8,2) createdAt : DATETIME Methods: createBooking() · updateBooking() · cancelBooking() · completeTrip() · getBookingById()	Customer (Many-to-One) Driver (Many-to-One) Vehicle (Many-to-One) Schedule (One-to-One) Invoice (One-to-One) Notification (One-to-Many)
Vehicle	vehicleId : INT (PK, AUTO_INCREMENT) registrationNumber : VARCHAR(20) make : VARCHAR(50) model : VARCHAR(50) capacity : INT status : ENUM('available','in-use','maintenance') Methods: assignToBooking() · checkAvailability() · getVehicleSchedule()	Booking (One-to-Many) Schedule (One-to-Many)
Schedule	scheduleId : INT (PK, AUTO_INCREMENT) bookingId : INT (FK → bookings) driverId : INT (FK → drivers) vehicleId : INT (FK → vehicles) scheduledStart : DATETIME scheduledEnd : DATETIME conflictFlag : TINYINT(1) DEFAULT 0 Methods: createEntry() · detectConflict() · getByDateRange() · updateStatus()	Booking (One-to-One) Driver (Many-to-One) Vehicle (Many-to-One)
Invoice	invoiceId : INT (PK, AUTO_INCREMENT) bookingId : INT (FK → bookings, UNIQUE) customerId : INT (FK → customers) invoiceDate : DATETIME subtotal : DECIMAL(8,2) taxAmount : DECIMAL(8,2) totalAmount : DECIMAL(8,2)	Booking (One-to-One) Customer (Many-to-One)

	pdfPath : VARCHAR(255) Methods: generateInvoice() · exportToPDF() · getInvoiceByBooking()	
Notification	notificationId : INT (PK, AUTO_INCREMENT) recipientType : ENUM('customer','driver') recipientId : INT channel : ENUM('sms','email') messageBody : VARCHAR(500) sentAt : DATETIME status : ENUM('logged','failed') Methods: logNotification() · getByRecipient() · getNotificationLog()	Customer (Many-to-One) Driver (Many-to-One) Booking (Many-to-One)

4.2 Domain Class Diagram

The domain class diagram for the NBK Travel Shuttle Booking Management System is presented in **Appendix B (Figure B.1)**. The diagram models the following relationships (IIE, 2026):

- **Inheritance (Generalisation):** The abstract class *Person* is the superclass for both *Customer* and *Driver*, sharing *fullName*, *phoneNumber*, and *emailAddress*. Both subclasses inherit from *Person* and extend it with role-specific attributes and methods.
- **Association — Customer to Booking:** One-to-Many (1 — 0..*). One customer can have many bookings; each booking belongs to exactly one customer.
- **Association — Driver to Booking:** One-to-Many (1 — 0..*). One driver can be assigned to many bookings over time.
- **Association — Vehicle to Booking:** One-to-Many (1 — 0..*). One vehicle can appear in many bookings.
- **Association — Booking to Schedule:** One-to-One (1 — 0..1). Each booking generates at most one schedule entry.
- **Association — Booking to Invoice:** One-to-One (1 — 0..1). Each completed booking generates at most one invoice.
- **Association — Booking to Notification:** One-to-Many (1 — 0..*). Each booking can trigger multiple notification log entries.
- **Dependency:** Booking «uses» Schedule for conflict detection during driver and vehicle assignment.

Figure B.1 — Domain Class Diagram (see Appendix B)

8 classes: «abstract» Person → Customer, Driver · Booking · Vehicle · Schedule · Invoice · Notification
Inheritance: Person ← Customer, Person ← Driver
Associations with cardinality as specified in Section 4.2
Diagram produced in draw.io — /docs/diagrams/class-diagram.drawio
GitHub: https://github.com/mzamon/Shuttle_Management_System/

Appendix A — UML Use Case Diagram

Figure A.1: UML Use Case Diagram — NBK Travel Shuttle Booking Management System

Figure A.1 — UML Use Case Diagram

Insert draw.io use case diagram image here.

File location: /docs/diagrams/use-case-diagram.drawio

GitHub: https://github.com/mzamon/Shuttle_Management_System/

Actors: Administrator (left) | Driver (left-bottom) | Customer (right) | System — Automated (right-bottom)

System boundary: "NBK Travel — Shuttle Booking Management System"

Subsystems: Booking Management | Scheduling System | Customer Records | Reporting Module | Invoice Generator | Notification Simulation

Use Cases: UC01 Create Booking | UC02 View/Edit/Cancel Booking | UC03 Assign Driver & Vehicle | UC04 Detect Conflict | UC05 View Schedule | UC06 Mark Trip Complete | UC07 Manage Customer Profile | UC08 View Booking History | UC09 Trip Report | UC10 Revenue Report | UC11 Top Customers | UC12 Driver Utilisation | UC13 Generate Invoice PDF | UC14 Log Notification

Relationships: UC03 «includes» UC04 | UC07 «extends» UC08 | UC09 «extends» UC10 | UC13 «includes» UC14

Appendix B — Domain Class Diagram

Figure B.1: Domain Class Diagram — NBK Travel Shuttle Booking Management System

Figure B.1 — Domain Class Diagram

Insert draw.io class diagram image here.

File location: /docs/diagrams/class-diagram.drawio

GitHub: https://github.com/mzamon/Shuttle_Management_System/

Classes: «abstract» Person (top-centre) | Customer (left, inherits Person) | Driver (right, inherits Person) | Booking (centre) | Vehicle (far right) | Schedule (bottom-centre) | Invoice (bottom-left) | Notification (bottom-right)

Relationships: Inheritance: Person ← Customer | Person ← Driver | Associations with cardinality per Section 4.2 | «uses» dependency from Booking → Schedule

Appendix C — References

draw.io (2024). *draw.io — diagram software and flowchart maker*. [Online] Available at: <https://www.draw.io> [Accessed 11 April 2026].

The Independent Institute of Education (Pty) Ltd (2026). *WIL Module Manual XISD5319w*. Johannesburg: IIE.

Project Management Institute (2021). *A guide to the project management body of knowledge (PMBOK Guide)* (7th ed.). Project Management Institute.

Document B

System Design

NBK Travel Shuttle Booking Management System

Table of Contents — System Design

1. Introduction	2
2. Logical Architectural Design — High-Level	2
2.1 Architecture Layer Descriptions	2
2.2 Component Interaction Flow	3
2.3 System Classification	3
3. Logical Architectural Design — Low-Level	4
4. User Interaction Design — Input Interactions	5
4.1 Input Menu Hierarchy	5
4.2 Input Form Specifications	6
5. User Interaction Design — Request Interactions	8
5.1 Request Interaction Summary	8
5.2 Report Parameter Forms	9
6. Database Design — Database Tables	10
7. Database Design — ERD Design	15
8. System Reports Design	16
Appendix A — High-Level Architecture Diagram	18
Appendix B — Low-Level Design Diagram	18
Appendix C — ERD Diagrams	19
Appendix D — System Prototype Screenshots	19
Appendix E — References	20

System Design — NBK Travel Shuttle Booking Management System

1. Introduction

This document presents the complete System Design for the **NBK Travel Shuttle Booking Management System** — a web-based platform commissioned to replace NBK Travel's manual, paper-based booking operations with a centralised, digital management solution. The design continues directly from the Requirement Analysis document and translates all functional requirements, use cases, and class relationships into a concrete, implementable technical specification (IIE, 2026).

System Purpose: The Shuttle Booking Management System enables NBK Travel's administrator to create and manage shuttle bookings, assign drivers and vehicles with automated conflict detection, maintain a structured customer database, and generate operational reports — all within a single, browser-accessible web interface. Drivers access a dedicated dashboard to view assigned trips and mark trips as completed.

Technical Overview: The system is implemented as a **three-tier web application**:

- **Presentation Layer:** Plain HTML5, CSS3, and JavaScript (ES6) — ensuring browser compatibility and zero licensing cost.
- **Business Logic Layer:** PHP 8.x backend with MySQLi prepared statements for all database interactions.
- **Data Layer:** MySQL Community Edition managed via phpMyAdmin on a WAMP Server local environment.

Business Goals:

- Eliminate double-bookings through real-time automated conflict detection.
- Centralise all NBK Travel operational data in a single, structured database.
- Enable management to access business intelligence reports on demand.
- Automate invoice generation and notification logging to reduce administrative workload.

Expected Impact: Post-deployment, NBK Travel is expected to eliminate scheduling conflicts, reduce manual administrative effort by an estimated 15–20 hours per week, and achieve full cost recovery within 16–21 weeks of deployment based on labour cost savings alone (IIE, 2026).

2. Logical Architectural Design — High-Level

The NBK Travel Shuttle Booking Management System follows a **Three-Tier (3-Tier) Layered Architecture**. This architecture separates the system into three distinct functional layers: the Presentation Layer (client), the Business Logic Layer (server), and the Data Layer (database). The high-level architecture diagram is presented in **Appendix A (Figure A.1)** (IIE, 2026).

2.1 Architecture Layer Descriptions

Tier 1 — Presentation Layer (Client / Browser) Technologies: HTML5 · CSS3 · JavaScript (ES6) · Chart.js (CDN) · html2pdf.js (CDN) Type: Thin client — all rendering in the user's browser Functions allocated here: Input form rendering and client-side validation · AJAX requests to PHP endpoints · Chart.js report rendering · html2pdf.js PDF generation · Session-based navigation and role-based menu display
↕ HTTP Request / Response (form submit / AJAX fetch / JSON)
Tier 2 — Business Logic Layer (WAMP Server — Apache + PHP 8.x) Technologies: PHP 8.x · Apache (WAMP) · MySQLi prepared statements · REST-aligned endpoints Functions allocated here: Authentication and session management · Booking CRUD operations · Driver and vehicle assignment with conflict detection · Customer record management · Report query execution and data aggregation · Notification simulation and logging · Invoice data assembly
↕ MySQLi Prepared Statement (SELECT / INSERT / UPDATE / DELETE) / ResultSet
Tier 3 — Data Layer (MySQL Database / phpMyAdmin) Technologies: MySQL Community Edition · phpMyAdmin Tables: bookings · customers · drivers · vehicles · schedules · invoices · notifications · users Note: The Presentation Layer never communicates directly with the database — all data access is mediated exclusively through the PHP Business Logic Layer.

2.2 Component Interaction Flow

Request flow (example: Create New Booking): 1. Administrator completes the New Booking Form in the browser (Tier 1) 2. Browser sends HTTP POST request to <i>createBooking.php</i> endpoint (Tier 2) 3. PHP validates input, executes MySQLi INSERT into <i>bookings</i> table, calls <i>logNotification()</i> (Tier 3) 4. MySQL returns success; PHP triggers <i>logNotification()</i> INSERT into <i>notifications</i> table 5. PHP returns JSON success response to browser 6. JavaScript renders booking confirmation card in the UI (Tier 1)
--

2.3 System Classification

Table 2.1: System Technical Classification

Property	Value
Architecture type	3-Tier Layered Architecture
Client type	Thin client (browser-based — no client-side installation required)
Application server	WAMP Server (Windows, Apache, MySQL, PHP)
Database position	Tier 3 — Data Layer, fully isolated from client
Authentication	PHP session-based (server-side); bcrypt password hashing
API style	REST-aligned PHP endpoints; JSON responses
Report rendering	Chart.js (CDN) for charts; html2pdf.js (CDN) for PDF invoices
Hosting (v1.0)	Local WAMP Server; architecture is cloud-deployment ready
Version control	GitHub — github.com/mzamon/Shuttle_Management_System

3. Logical Architectural Design — Low-Level

The low-level design models the precise internal relationships between actors, PHP functions (use cases), and MySQL database tables. It consists of three components: (1) Actors from the use case diagrams, (2) PHP functions with «include» and «extend» relationships indicated by arrows, and (3) MySQL tables annotated for each use case. The Low-Level Design Diagram is presented in **Appendix B (Figure B.1)** (IIE, 2026).

Table 3.1: Actor — Function — Database Mapping (Low-Level)

Actor	PHP Function (Use Case)	Relationship	MySQL Table(s)
Administrator	UC01 — createBooking(\$params)	—	bookings, customers
Administrator	UC02 — updateBooking() / cancelBooking()	—	bookings
Administrator	UC03 — assignDriverVehicle(\$id,...)	«includes» UC04	schedules, drivers, vehicles
System (Automated)	UC04 — detectConflict(\$driverId,...)	Included by UC03	schedules
Administrator	UC05 — getSchedule(\$start, \$end)	—	schedules, bookings
Driver	UC06 — completeTrip(\$bookingId, \$driverId)	—	bookings, schedules
Administrator	UC07 — createOrUpdateCustomer()	«extends» UC08	customers
System (Automated)	UC08 — matchCustomerRecord(\$name, \$phone)	Extends UC07	customers, bookings
Administrator	UC09 — generateTripReport(\$s, \$e, \$groupBy)	«extends» UC10	bookings
Administrator	UC10 — generateRevenueReport(\$s, \$e)	—	bookings
Administrator	UC11 — getTopCustomers(\$limit)	—	customers, bookings
Administrator	UC12 — getDriverStats(\$driverId, \$s, \$e)	—	schedules, bookings
Administrator	UC13 — generateInvoice(\$bookingId)	«includes» UC14	bookings, customers, invoices
System (Automated)	UC14 — logNotification(\$params)	Included by UC13	notifications
Driver	getAssignedTrips(\$driverId)	—	schedules, bookings
Administrator / Driver	authenticateUser(\$username, \$password)	—	users

4. User Interaction Design — Input Interactions

Input interactions represent all data controls used to read data into the NBK Travel Shuttle Booking Management System. All input menus, forms, fields, data types, and validation rules are specified below using a structured annotated menu list (Option 1) as specified in the module requirements (IIE, 2026). System prototype screenshots are provided in Appendix D.

4.1 Input Menu Hierarchy

Table 4.1: Input Menu Hierarchy — NBK Travel Shuttle Booking Management System

Menu Level	Module	Menu Item	Sub-Item 1	Sub-Item 2	Sub-Item 3
1st Level	Booking Management				
2nd Level	New Booking Form				
3rd Level			Customer Name	Pickup Location	Drop-off Location
3rd Level			Booking Date	Booking Time	No. of Passengers
3rd Level			Fare Amount (ZAR)	Status	
2nd Level	Manage Bookings				
3rd Level			Filter by Date	Filter by Status	View Detail
3rd Level			Edit Booking	Cancel Booking	
1st Level	Schedule Management				
2nd Level	Assign Driver & Vehicle				
3rd Level			Select Booking ID	Select Driver	Select Vehicle
3rd Level			Scheduled Start	Scheduled End	Conflict Flag (system)
2nd Level	View Schedule				
3rd Level			Select Date (Daily)	Select Week (Weekly)	
1st Level	Customer Records				
2nd Level	New Customer Form				
3rd Level			Full Name	Phone Number	Email Address
3rd Level			Travel Preferences		
2nd Level	Search Customer				
3rd Level			Name / Phone Search	View History	Edit Profile
1st Level	Reports				
2nd Level	Trip Report		Start Date	End Date	Group By
2nd Level	Revenue Report		Start Date	End Date	Output Format
2nd Level	Top Customers		Top N (limit)		
2nd Level	Driver Utilisation		Driver Select	Start Date	End Date
1st Level	Invoice Generator				
2nd Level	Select Completed Booking		Generate PDF (html2pdf.js)		
1st Level	Driver Dashboard				
2nd Level	View Assigned Trips		Mark Trip Complete		
1st Level	Authentication				
2nd Level	Login Form		Username	Password	Login Button

4.2 Input Form Specifications

4.2.1 New Booking Form

Table 4.2: New Booking Form Fields

Field Name	Data Type (PHP/MySQL)	Validation	Implementation Notes
Customer Name	VARCHAR(100)	Required; min 2 characters	AJAX auto-suggest from customers table; creates new record if no match found
Pickup Location	VARCHAR(100)	Required	Free-text with optional predefined NBK Travel route dropdown
Drop-off Location	VARCHAR(100)	Required	Free-text address field
Booking Date	DATE	Required; must not be in the past	HTML5 date picker; PHP server-side re-validation
Booking Time	TIME	Required	HTML5 time picker; 24-hour format
No. of Passengers	INT	Required; min 1, max 50	HTML number input with min/max; PHP range check
Fare Amount (ZAR)	DECIMAL(8,2)	Required; must be > 0	PHP regex: <code>/^\d+(\.\d{1,2})?\$/</code>
Status	ENUM	Default: 'pending'	SELECT dropdown: pending/confirmed/cancelled/completed

4.2.2 New / Edit Customer Form

Table 4.3: Customer Form Fields

Field Name	Data Type	Validation	Notes
Full Name	VARCHAR(100)	Required; min 2 characters	Trimmed on input; stored in Title Case
Phone Number	VARCHAR(20)	Required; exactly 10 digits	PHP regex: <code>/^0[0-9]{9}\$/</code> ; format: 0XXXXXXXXX
Email Address	VARCHAR(100)	Optional; valid email format	PHP FILTER_VALIDATE_EMAIL; used for notification simulation
Travel Preferences	VARCHAR(255)	Optional	Free-text; e.g. preferred driver, vehicle type, billing notes

4.2.3 Assign Driver & Vehicle Form

Table 4.4: Schedule Assignment Form Fields

Field Name	Data Type	Validation	Notes
Booking Reference	INT (FK)	Required; must exist in bookings	Auto-populated from booking selection dropdown
Driver	INT (FK)	Required; status = 'available'	PHP SELECT filtered to available drivers only
Vehicle	INT (FK)	Required; status = 'available'	PHP SELECT filtered to available vehicles only
Scheduled Start	DATETIME	Required	Auto-populated from booking date and time
Scheduled End	DATETIME	Required	Calculated from estimated trip duration
Conflict Flag	TINYINT(1)	System-set	detectConflict() sets to 1 if overlap found; form submission blocked with red alert modal

4.2.4 Login Form

Table 4.5: Authentication Form Fields

Field Name	Data Type	Validation	Notes
Username	VARCHAR(50)	Required; must exist in users table	Trimmed; case-insensitive lookup
Password	VARCHAR(255)	Required; min 8 characters	Compared against bcrypt hash via password_verify(); never stored plain-text

5. User Interaction Design — Request Interactions

Request interactions represent all service requests placed on the NBK Travel system that produce functional outputs — including screen displays, printable reports, and PDF downloads. These interactions are triggered by user selections in menus and parameter forms (IIE, 2026).

5.1 Request Interaction Summary

Table 5.1: Request Interactions — NBK Travel Shuttle Booking Management System

Request / Output	Triggered By	Output Format	MySQL Data Source	PHP Method
Booking List View	Admin selects View Bookings	Screen — HTML table with pagination and filters	bookings JOIN customers	getAllBookings(\$filters)
Booking Detail View	Admin clicks a booking row	Screen — PHP detail card	bookings + customers + drivers + vehicles	getBookingById(\$bookingId)
Schedule Grid (Daily)	Admin selects a date	Screen — HTML/CSS daily calendar grid	schedules JOIN bookings JOIN drivers	getSchedule(\$date, \$date)
Schedule Grid (Weekly)	Admin selects a week	Screen — HTML/CSS weekly grid	schedules JOIN bookings JOIN drivers	getSchedule(\$start, \$end)
Conflict Alert	detectConflict() on assignment	Screen — red JavaScript modal; assignment blocked	schedules table	detectConflict(\$driverId, \$vehicleId, \$datetime, \$duration)
Customer Profile & History	Admin searches by name or phone	Screen — profile card + booking history list	customers JOIN bookings	searchCustomer(\$query)
Trip Count Report	Admin selects date range and group by	Screen — Chart.js bar/line + print view	bookings (bookingDate, status)	generateTripReport(\$start, \$end, \$groupBy)
Revenue Summary Report	Admin selects period	Screen — Chart.js bar chart + print/PDF	bookings.fareAmount grouped by period	generateRevenueReport(\$start, \$end)
Top Customers Report	Admin selects limit (N)	Screen — HTML ranked table	customers JOIN bookings COUNT(*)	getTopCustomers(\$limit)
Driver Utilisation Report	Admin selects driver and period	Screen — HTML table (trips and hours per driver)	schedules JOIN bookings WHERE driverId	getDriverStats(\$driverId, \$start, \$end)
Invoice / Receipt (PDF)	Admin clicks Generate Invoice	PDF download via html2pdf.js	bookings + customers + invoices	generateInvoice(\$bookingId)
Notification Audit Log	Admin selects Notification Log	Screen — read-only HTML log table	notifications table	getNotificationLog(\$start, \$end)
Booking Status Dashboard	Admin logs in	Screen — metric cards + Chart.js summary	bookings, schedules, drivers, vehicles	getDashboardMetrics()
Driver Trip List	Driver logs in to dashboard	Screen — driver's assigned trips	schedules JOIN bookings WHERE driverId	getAssignedTrips(\$driverId)

5.2 Report Parameter Forms

Table 5.2: Trip Report Parameter Form

Field	Type	Validation	Purpose
Start Date	DATE	Required	Filter period start
End Date	DATE	Required; must be $\geq$ Start Date	Filter period end
Group By	ENUM (Day/Week/Month)	Required	Chart granularity for Chart.js rendering
Output	ENUM (Screen/Print)	Default: Screen	Print triggers window.print() with CSS print media queries

Table 5.3: Revenue Report Parameter Form

Field	Type	Validation	Purpose
Start Date	DATE	Required	Filter period start
End Date	DATE	Required; must be $\geq$ Start Date	Filter period end
Currency	VARCHAR(10)	Default: ZAR	Display formatting
Output	ENUM (Screen/Print/PDF)	Default: Screen	PDF via html2pdf.js

Table 5.4: Driver Utilisation Report Parameter Form

Field	Type	Validation	Purpose
Driver	INT (FK)	Required; select from drivers table	Filter by specific driver
Start Date	DATE	Required	Filter period start
End Date	DATE	Required; must be $\geq$ Start Date	Filter period end
Output	ENUM (Screen/Print)	Default: Screen	Rendering target

6. Database Design — Database Tables

The database design is derived directly from the Class Diagram (Requirement Analysis, Section 4) and the Logical System Model (Section 3). Each entity class becomes a MySQL table, implemented via phpMyAdmin as part of the Database Development task (T5). All tables are normalised to at least Third Normal Form (3NF) to eliminate redundancy and ensure data integrity (Connolly and Begg, 2015).

DBMS: MySQL Community Edition — phpMyAdmin / WAMP Server | **Backend:** PHP 8.x + MySQLi | **Normalisation:** 3NF | **GitHub:** github.com/mzamon/Shuttle_Management_System

Database Table 1: bookings

Table 6.1: bookings

Primary Key	FK #1	FK #2	FK #3	Data Fields
bookingId INT (PK, AI)	customerId INT (FK → customers)	driverId INT (FK → drivers)	vehicleId INT (FK → vehicles)	pickupLocation VARCHAR(100) dropoffLocation VARCHAR(100) bookingDate DATETIME passengers INT status ENUM('pending','confirmed','completed','cancelled') cancellationReason VARCHAR(255) fareAmount DECIMAL(8,2) createdAt DATETIME

Table 6.1a: bookings — Sample Data

bookingId	customerId	driverId	vehicleId	pickupLocation	dropoffLocation	bookingDate	passengers	status	fareAmount
1	3	2	1	14 Main Rd, Johannesburg	Bela-Bela	2026-06-01 07:30	3	confirmed	R180.00
2	1	1	3	Sandton City	Mossel Bay	2026-06-02 09:00	1	confirmed	R450.00
3	5	3	2	Midrand	Bloemfontein	2026-06-03 06:45	2	pending	R620.00

Database Table 2: customers

Table 6.2: customers — Sample Data

customerId (PK)	fullName	phoneNumber	emailAddress	preferences	createdAt
1	Shenice Wood	0608178335	jessicawood07@icloud.com	Window seat preferred	2026-05-01 08:00
2	Zoe Mokoena	0712345678	ZM@nbktravel.co.za	Early morning pickup preferred	2026-05-02 09:30
3	Matt Maopye	0834567890	matt.n@work.co.za	Requires receipt always	2026-05-03 11:00

Database Table 3: drivers

Table 6.3: drivers — Sample Data

driverId (PK)	fullName	licenceNumber	phoneNumber	status
1	Joyce Dlamini	PDP-JHB12345	0761234567	available
2	Joe Khumalo	PDP-JHB67890	0829876543	on-trip
3	Maria Zamani	PDP-JHB24680	0734455667	available

Database Table 4: vehicles

Table 6.4: vehicles — Sample Data

vehicleId (PK)	registrationNumber	make	model	capacity	status
1	RY 12 RF GP	Toyota	Quantum	14	available
2	RXX 719 GP	Mercedes-Benz	Vito	8	in-use
3	ZWC 355 NW	Hyundai	H1	7	available

Database Table 5: schedules

Table 6.5: schedules — Sample Data

scheduleId (PK)	bookingId (FK)	driverId (FK)	vehicleId (FK)	scheduledStart	scheduledEnd	conflictFlag
1	1	2	1	2026-06-01 07:30	2026-06-01 09:00	0
2	2	1	3	2026-06-02 09:00	2026-06-02 09:45	0
3	3	3	2	2026-06-03 06:45	2026-06-03 08:30	0

Database Table 6: invoices

Table 6.6: invoices — Sample Data

invoicId (PK)	bookingId (FK, UNIQUE)	customerId (FK)	invoiceDate	subtotal	taxAmount	totalAmount	pdfPath
1	2	1	2026-06-02 10:05	R156.52	R23.48	R180.00	/invoices/INV001.pdf
2	5	3	2026-06-05 14:30	R391.30	R58.70	R450.00	/invoices/INV002.pdf
3	7	2	2026-06-07 09:15	R538.26	R81.74	R620.00	/invoices/INV003.pdf

Database Table 7: notifications

Table 6.7: notifications — Sample Data

notifId (PK)	recipientType	recipientId	channel	messageBody	sentAt	status
1	customer	1	email	Booking #1 confirmed for 2026-06-01 07:30. Thank you for choosing NBK Travel.	2026-05-28 14:22	logged
2	driver	2	email	Trip #1 assigned. Pickup: 14 Main Rd, JHB at 07:30. Please confirm availability.	2026-05-28 14:22	logged
3	customer	3	email	Invoice INV001 generated for your completed trip.	2026-06-02 10:06	logged

Database Table 8: users

Table 6.8: users — Sample Data

userId (PK)	username (UNIQUE)	passwordHash	role	createdAt
1	admin	\$2y\$10\$examplehash...	admin	2026-01-01 08:00
2	joyce.dlamini	\$2y\$10\$examplehash...	driver	2026-01-05 09:00
3	joe.khumalo	\$2y\$10\$examplehash...	driver	2026-01-05 09:00

7. Database Design — ERD Design

The Entity Relationship Diagrams below model the structural relationships between all MySQL tables in the NBK Travel Shuttle Booking Management System. Standard crow's foot notation is used — rectangles for tables, lines for relationships, crow's foot symbols for cardinality. All ERD diagrams were produced using draw.io and are stored at `/docs/diagrams/` in the GitHub repository (IIE, 2026). Full diagrams are presented in **Appendix C (Figures C.1 and C.2)**.

ERD Diagram 1 — Core Transaction Entities (Figure C.1)

This ERD models the primary transactional relationships between the bookings, customers, drivers, vehicles, and schedules tables.

Table 7.1: ERD 1 Relationship Summary

From Table	Relationship	To Table	Cardinality
CUSTOMERS	places	BOOKINGS	1 to 0..*
DRIVERS	assigned to	BOOKINGS	1 to 0..*
VEHICLES	used in	BOOKINGS	1 to 0..*
BOOKINGS	scheduled via	SCHEDULES	1 to 0..1
DRIVERS	appears in	SCHEDULES	1 to 0..*
VEHICLES	appears in	SCHEDULES	1 to 0..*

ERD Diagram 2 — Supporting Entities (Figure C.2)

This ERD models the post-trip financial and communication record-keeping relationships between bookings, invoices, notifications, and customers.

Table 7.2: ERD 2 Relationship Summary

From Table	Relationship	To Table	Cardinality
BOOKINGS	generates	INVOICES	1 to 0..1
BOOKINGS	triggers	NOTIFICATIONS	1 to 0..*
CUSTOMERS	billed via	INVOICES	1 to 0..*
CUSTOMERS	notified by	NOTIFICATIONS	1 to 0..*

Figure C.1 — ERD: Core Transaction Entities & Figure C.2 — ERD: Supporting Entities (see Appendix C)

Diagrams produced in draw.io using crow's foot notation
Files: `/docs/diagrams/erd-core.drawio` and `/docs/diagrams/erd-supporting.drawio`
GitHub: https://github.com/mzamon/Shuttle_Management_System/

8. System Reports Design

The following reports have been identified as required system outputs for NBK Travel management. Each report specifies the data source, presentation format, intended audience, PHP method, and business value. All report queries are optimised with indexes on *bookingDate*, *customerId*, and *driverId* for acceptable performance as NBK Travel's booking volume scales (IIE, 2026).

Table 8.1: System Reports Design — NBK Travel Shuttle Booking Management System

#	Report Name	Description	MySQL Data Source	Output Format	PHP Method	Business Value
R01	Trip Count Report	Count of trips per day, week, or month for a selected date range	bookings (bookingDate, status)	Screen — Chart.js bar/line chart + printable HTML table	generateTripReport(\$start, \$end, \$groupBy)	Identifies peak booking periods; supports fleet capacity planning
R02	Revenue Summary Report	Total fare income grouped by day, week, or month	bookings.fareAmount grouped by date	Screen — Chart.js bar chart + printable table + PDF via html2pdf.js	generateRevenueReport(\$start, \$end)	Accurate revenue tracking and trend analysis for management decisions
R03	Top Customers Report	Ranked list by booking frequency with total spend	customers JOIN bookings COUNT(*)	Screen — HTML ranked table	getTopCustomers(\$limit)	Identifies high-value clients for relationship management
R04	Driver Utilisation Report	Trips and total hours per driver in a selected period	schedules JOIN bookings WHERE driverId	Screen — HTML table per driver	getDriverStats(\$driverId, \$start, \$end)	Enables fair workload distribution; identifies over- or under-utilised drivers
R05	Invoice / Receipt (PDF)	Itemised PDF receipt per completed booking with fare, tax, and NBK Travel branding	bookings + customers + invoices	PDF download via html2pdf.js	generateInvoice(\$bookingId)	Professional automated billing; supports dispute resolution
R06	Notification Audit Log	Full read-only log of all simulated SMS and email notifications	notifications table	Screen — read-only HTML log table	getNotificationLog(\$start, \$end)	Verifiable audit trail of all client and driver communications
R07	Booking Status Summary	Count of bookings by status (pending, confirmed, completed, cancelled) for a selected period	bookings (status, bookingDate)	Screen — Chart.js pie/doughnut chart	getBookingStatusSummary(\$start, \$end)	Operational health overview — proportion of completed vs cancelled bookings
R08	Daily Operations Dashboard	Live summary of today's bookings, active drivers, available vehicles, and revenue-to-date	bookings, schedules, drivers, vehicles	Screen — dashboard metric cards + Chart.js summary charts	getDashboardMetrics()	Real-time operational overview for NBK Travel management on login

8.1 Report Design Notes

- All screen-based reports use **Chart.js** (loaded via CDN) for graphical rendering within the HTML/CSS/JS frontend.
- Print views are triggered via the browser's native `window.print()` dialog with print-specific CSS media queries hiding navigation and admin controls.
- PDF invoices (R05)** are generated client-side using `html2pdf.js` (CDN), converting the rendered HTML invoice template into a downloadable PDF — a zero-cost, zero-server-dependency approach requiring no additional PHP libraries.

- All report queries use MySQLi prepared statements, preventing SQL injection and ensuring data integrity.
- Report access is restricted to the Administrator role via PHP session authentication; drivers access only their own trip data through the driver dashboard (IIE, 2026).

Appendix A — High-Level Architecture Diagram

Figure A.1: High-Level 3-Tier Architecture — NBK Travel Shuttle Booking Management System

Figure A.1 — High-Level 3-Tier Architecture Diagram

Insert draw.io 3-tier architecture diagram here.

File location: /docs/diagrams/architecture-high-level.drawio

GitHub: https://github.com/mzamon/Shuttle_Management_System/

Three horizontal layers:

Layer 1 (blue) — Presentation Layer: HTML5 · CSS3 · JavaScript (ES6) · Chart.js (CDN) · html2pdf.js (CDN) | Label: "Thin Client — Browser"

Layer 2 (green) — Business Logic Layer: PHP 8.x · Apache (WAMP) · MySQLi · REST endpoints · Session management | Label: "Application Server — WAMP/Apache/PHP 8.x"

Layer 3 (amber) — Data Layer: MySQL · phpMyAdmin · 8 tables: bookings, customers, drivers, vehicles, schedules, invoices, notifications, users | Label: "MySQL Community Edition — phpMyAdmin"

Arrows between layers: HTTP Request/Response (L1↔L2) | MySQLi Prepared Statement / ResultSet (L2↔L3)

Appendix B — Low-Level Design Diagram

Figure B.1: Low-Level Design Diagram — NBK Travel Shuttle Booking Management System

Figure B.1 — Low-Level Design Diagram

Insert draw.io low-level design diagram here.

File location: /docs/diagrams/architecture-low-level.drawio

GitHub: https://github.com/mzamon/Shuttle_Management_System/

Three columns: Actors (left) | PHP functions/use cases in dashed system boundary (centre) | MySQL table ovals (right)

Actors: Administrator | Driver | Customer | System (Automated)

Functions: UC01 createBooking → UC14 logNotification + authenticateUser + getAssignedTrips

Tables: bookings · schedules · customers · invoices · notifications · users

Relationships: «include» UC03→UC04, UC13→UC14 | «extend» UC07→UC08, UC09→UC10

Appendix C — ERD Diagrams

Figure C.1: ERD — Core Transaction Entities (*bookings, customers, drivers, vehicles, schedules*)

Figure C.1 — ERD: Core Transaction Entities

Insert draw.io ERD diagram here — `/docs/diagrams/erd-core.drawio`
Notation: crow's foot | Cardinality: 1 to 0..* and 1 to 0..1 as per Table 7.1

Figure C.2: ERD — Supporting Entities (*bookings, invoices, notifications, customers*)

Figure C.2 — ERD: Supporting Entities

Insert draw.io ERD diagram here — `/docs/diagrams/erd-supporting.drawio`
Notation: crow's foot | Cardinality: 1 to 0..1 and 1 to 0..* as per Table 7.2

Appendix D — System Prototype Screenshots

Screenshots from the working NBK Travel Shuttle Booking Management System prototype — stored at `/docs/prototype/` in the GitHub repository.

Table D.1: Prototype Screenshot Index

Figure	Screen	Description
D.1	Dashboard	Operational overview with total bookings, customers, revenue metric cards, and recent bookings table
D.2	New Booking Form	Full booking creation form with all fields and Create Booking button
D.3	All Bookings Table	Paginated, sortable bookings table with status badges and Cancel action buttons
D.4	Schedule & Assignment	Assign Driver & Vehicle form with conflict detection result and weekly schedule grid
D.5	Customer Records	Customer directory with name/phone search and booking count per customer
D.6	Reports	Trip and Revenue charts (Chart.js) with date range selectors and Top Customers table
D.7	Invoice Generator	Invoice PDF preview with booking selector and Download PDF Invoice button

Insert actual prototype screenshots here. Screenshots are stored in the GitHub repository at:
https://github.com/mzamon/Shuttle_Management_System/tree/main/docs/prototype/

Appendix E — References

- Connolly, T. and Begg, C. (2015). *Database systems: a practical approach to design, implementation, and management* (6th ed.). Pearson Education.
- draw.io (2024). *draw.io — diagram software and flowchart maker*. [Online] Available at: <https://www.draw.io> [Accessed 11 April 2026].
- The Independent Institute of Education (Pty) Ltd (2026). *WIL Module Manual XISD5319w*. Johannesburg: IIE.
- Project Management Institute (2021). *A guide to the project management body of knowledge (PMBOK Guide)* (7th ed.). Project Management Institute.